

DBMS Project Report

Project Title: Stock Portfolio and Trading Management System

Course Name & Code: UCS310 – Database Management Systems

Degree & Year: B.Tech (2nd Year)

Department: Computer Engineering

Institute Name: Thapar Institute of Engineering & Technology

Group Members:

Avneet Kaur (Roll No: _____)

Archita Sharma (Roll No: _____)

Shrey Singh (Roll No: _____)

Lab Instructor: _____

Academic Year: 2025–26

Table of Contents

1. Introduction 3

2. Problem Statement 3

3. Objectives of the Project 3

4. Scope of the Project 4

5. Proposed System Description 4

6. Database Design 5

   6.1 Entities Identified 5

   6.2 ER Diagram 5

   6.3 Relational Schema 6

7. Normalization 8

8. Database Implementation 9

   8.1 SQL Implementation 9

   8.2 PL/SQL Components 11

9. Sample SQL Queries 14

10. Tools & Technologies Used 15

11. Expected Outcomes 15

1. Introduction

The Stock Portfolio and Trading Management System is a database-driven application designed to manage investors, stocks, portfolios, trade transactions, and holdings efficiently. The system stores and manages investor records, stock listings, portfolio details, buy/sell transactions, and real-time holdings information.

Traditionally, many small-scale brokerage operations and individual investors track their portfolios manually using spreadsheets or paper records. This file-based approach suffers from data redundancy, inconsistency, delayed updates, and lack of audit trails. A Database Management System (DBMS) resolves these issues by providing structured data storage, integrity constraints (such as preventing oversell of shares), and efficient query processing through indexes and views.

This project emphasizes backend implementation using Oracle SQL and advanced PL/SQL constructs, demonstrating procedures, triggers, cursors, and views to automate trading operations and portfolio management.

2. Problem Statement

In many small brokerage firms and individual investor setups, operations such as recording trades, tracking holdings, and computing portfolio values are handled manually. This leads to:

- Duplicate or outdated investor and stock records
- Inaccurate holdings tracking after frequent buy/sell transactions
- Difficulty in preventing oversell scenarios where investors attempt to sell more shares than they hold
- Slow data retrieval for portfolio summaries and profit/loss reports
- Lack of audit trails for administrative oversight of trading activity

The proposed Stock Portfolio and Trading Management System provides a structured relational database solution to manage investors, stocks, portfolios, transactions, and holdings efficiently, ensuring reliable, consistent, and secure operations.

3. Objectives of the Project

- To design a database using an Entity-Relationship (E-R) data model.
- To convert the ER model into relational tables with strict foreign key relationships.
- To normalize the database up to Third Normal Form (3NF).
- To implement DDL and DML SQL commands.
- To demonstrate advanced PL/SQL constructs like stored procedures, triggers, and cursors.
- To ensure data consistency using check constraints, sequences, and transactions (ACID properties).
- To maintain audit logs for all trading transactions through automated triggers.

4. Scope of the Project

Functional Boundaries: The system manages the complete lifecycle of trading activity, from investor registration and portfolio creation to recording buy/sell transactions, automatic holdings updates, and generation of portfolio summary reports.

Types of Users involved:

- Admin/Staff (Brokerage personnel managing investors and stocks)
- Investors (whose portfolios and trades are tracked)

Modules Covered:

- Investor Registration Module
- Stock Master Module
- Portfolio Creation Module
- Trade Transaction Module (Buy/Sell)
- Holdings Tracking Module
- Profit/Loss Reporting Module
- Admin Audit Log Module

5. Proposed System Description

The system is developed using Oracle SQL and PL/SQL for the database layer, with potential frontend integration through standard web technologies.

Working of the System:

1. Admin/staff registers investors with unique credentials (PAN, email, phone).
2. Stocks are pre-loaded into the system with their symbol, name, sector, and current market price.
3. Each investor is assigned a portfolio for organizing their holdings.
4. When an investor places a BUY or SELL order, a stored procedure (buy_stock / sell_stock) inserts a new record into the trade_transactions table.
5. A BEFORE INSERT trigger validates SELL orders to prevent overselling beyond available holdings.
6. An AFTER INSERT trigger automatically updates the holdings table — adding new rows or adjusting quantities for existing positions.
7. A separate trigger logs every transaction into the admin_log table for audit purposes.
8. A portfolio summary view (vw_portfolio_summary) joins investor, portfolio, holdings, and stock data to provide real-time market valuations.

This system greatly improves operational efficiency, data consistency, and traceability of all trading activity.

6. Database Design

6.1 Entities Identified

1. Investor
2. Stock
3. Portfolio
4. Trade Transactions
5. Holdings
6. Profit/Loss Report
7. Admin Log

6.2 ER Diagram

The Entity-Relationship diagram on the following page illustrates the entities, their attributes, and the relationships between them. Rectangles denote entities, ovals denote attributes (with primary keys underlined), and diamonds denote relationships with cardinality constraints (1, N) labeled on the connecting edges.

Key relationships depicted:

- Investor — Owns — Portfolio (1:1)
- Investor — Places — Trade Transactions (1:N)
- Stock — Involves — Trade Transactions (1:N)
- Portfolio — Contains — Holdings (1:N)
- Stock — Held In — Holdings (1:N)
- Investor — Has — Profit/Loss Report (1:N)
- Trade Transactions — Logged — Admin Log (1:N)

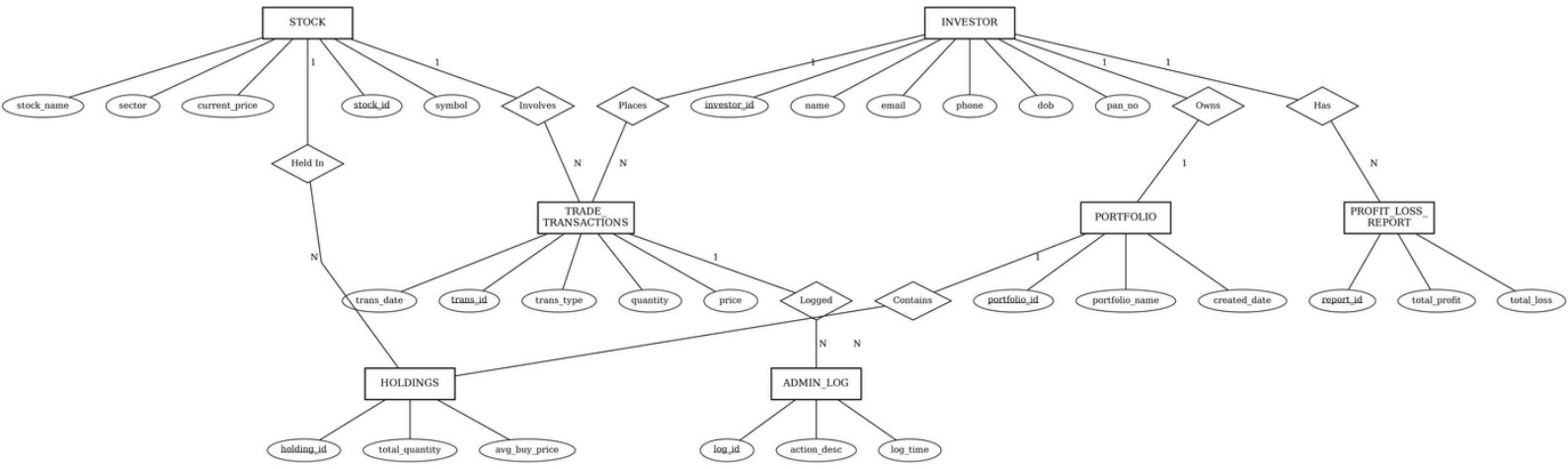


Figure 6.1 — Entity-Relationship diagram for the Stock Portfolio and Trading Management System.

6.3 Relational Schema

INVESTOR

```
CREATE TABLE investor (

    investor_id  NUMBER PRIMARY KEY,

    name         VARCHAR2(50) NOT NULL,

    email        VARCHAR2(60) UNIQUE NOT NULL,

    phone        VARCHAR2(15) UNIQUE,

    dob          DATE,

    pan_no       VARCHAR2(15) UNIQUE NOT NULL

);
```

STOCK

```
CREATE TABLE stock (

    stock_id       NUMBER PRIMARY KEY,

    symbol         VARCHAR2(10) UNIQUE NOT NULL,

    stock_name     VARCHAR2(60) NOT NULL,

    sector         VARCHAR2(30),

    current_price  NUMBER(12,2) CHECK (current_price > 0)

);
```

PORTFOLIO

```
CREATE TABLE portfolio (

    portfolio_id    NUMBER PRIMARY KEY,

    investor_id     NUMBER UNIQUE,

    portfolio_name  VARCHAR2(50),

    created_date    DATE DEFAULT SYSDATE,

    FOREIGN KEY (investor_id) REFERENCES investor(investor_id)
```

```
);
```

TRADE_TRANSACTIONS

```
CREATE TABLE trade_transactions (

    trans_id      NUMBER PRIMARY KEY,

    investor_id   NUMBER,

    stock_id      NUMBER,

    portfolio_id  NUMBER,

    trans_type    VARCHAR2(10) CHECK (trans_type IN ('BUY','SELL')),

    quantity      NUMBER CHECK (quantity > 0),

    price         NUMBER(12,2) CHECK (price > 0),

    trans_date    DATE DEFAULT SYSDATE,

    FOREIGN KEY (investor_id)  REFERENCES investor(investor_id),

    FOREIGN KEY (stock_id)     REFERENCES stock(stock_id),

    FOREIGN KEY (portfolio_id) REFERENCES portfolio(portfolio_id)

);
```

HOLDINGS

```
CREATE TABLE holdings (

    holding_id      NUMBER PRIMARY KEY,

    portfolio_id    NUMBER,

    stock_id        NUMBER,

    total_quantity  NUMBER DEFAULT 0 CHECK (total_quantity >= 0),

    avg_buy_price   NUMBER(12,2),

    FOREIGN KEY (portfolio_id) REFERENCES portfolio(portfolio_id),

    FOREIGN KEY (stock_id)     REFERENCES stock(stock_id)

);
```

PROFIT_LOSS_REPORT

```
CREATE TABLE profit_loss_report (

    report_id     NUMBER PRIMARY KEY,

    investor_id   NUMBER,

    stock_id      NUMBER,

    total_profit  NUMBER(12,2) DEFAULT 0,

    total_loss    NUMBER(12,2) DEFAULT 0,

    FOREIGN KEY (investor_id) REFERENCES investor(investor_id),

    FOREIGN KEY (stock_id)    REFERENCES stock(stock_id)

);
```

ADMIN_LOG

```
CREATE TABLE admin_log (

    log_id       NUMBER PRIMARY KEY,

    action_desc  VARCHAR2(200),

    log_time     DATE DEFAULT SYSDATE

);
```

7. Normalization

Functional Dependencies:

- Investor: investor_id → name, email, phone, dob, pan_no
- Stock: stock_id → symbol, stock_name, sector, current_price
- Portfolio: portfolio_id → investor_id, portfolio_name, created_date
- Trade Transactions: trans_id → investor_id, stock_id, portfolio_id, trans_type, quantity, price, trans_date
- Holdings: holding_id → portfolio_id, stock_id, total_quantity, avg_buy_price
- Profit/Loss Report: report_id → investor_id, stock_id, total_profit, total_loss
- Admin Log: log_id → action_desc, log_time

1NF (First Normal Form)

- All attributes are atomic (single-valued).
- Each table has a primary key and no repeating groups.

2NF (Second Normal Form)

- All tables use single-column primary keys, ensuring no partial dependency exists.
- Every non-key attribute depends fully on its table's primary key.

3NF (Third Normal Form)

- Stock details are stored once in the STOCK master table and referenced via foreign keys (stock_id) in transactions and holdings — eliminating redundancy of stock_name, symbol, and price across multiple tables.
- Investor details are similarly centralized and referenced through investor_id.
- All non-key attributes depend only on the primary key of their respective tables, eliminating transitive dependencies.

Conclusion: The database is successfully normalized up to 3NF.

8. Database Implementation

8.1 SQL Implementation

DDL and DML Commands Used

Sample data insertion into the Investor table:

```
INSERT INTO investor VALUES

  (101,'Avneet Kaur','avneet@gmail.com','9876500001',

   DATE '2005-03-12','PANAK101');

INSERT INTO investor VALUES

  (102,'Archita Sharma','archita@gmail.com','9876500002',

   DATE '2005-06-18','PANAS102');

INSERT INTO investor VALUES

  (103,'Shrey Singh','shrey@gmail.com','9876500003',

   DATE '2004-11-02','PANSS103');
```

Sample data insertion into the Stock table:

```
INSERT INTO stock VALUES

  (1,'TCS','Tata Consultancy Services','IT',3800);

INSERT INTO stock VALUES

  (2,'INFY','Infosys Ltd','IT',1450);

INSERT INTO stock VALUES

  (5,'HDFCBK','HDFC Bank','Banking',1620);

INSERT INTO stock VALUES

  (8,'RELI','Reliance Industries','Energy',2890);
```

Sample data insertion into the Portfolio table:

```
INSERT INTO portfolio VALUES
```

```
  (201,101,'Growth Portfolio',SYSDATE);

INSERT INTO portfolio VALUES

  (202,102,'Balanced Portfolio',SYSDATE);

INSERT INTO portfolio VALUES

  (203,103,'Long Term Wealth',SYSDATE);


COMMIT;
```

8.2 PL/SQL Components

The following PL/SQL components have been implemented to demonstrate advanced DBMS functionalities:

Stored Procedure — buy_stock:

Encapsulates the logic of inserting a BUY transaction by fetching the current market price of the stock and recording the trade in the trade_transactions table.

```
CREATE OR REPLACE PROCEDURE buy_stock(

    p_trans_id     NUMBER,

    p_investor_id  NUMBER,

    p_stock_id     NUMBER,

    p_portfolio_id NUMBER,

    p_qty          NUMBER

)

IS

    v_price NUMBER;

BEGIN

    SELECT current_price INTO v_price

    FROM stock

    WHERE stock_id = p_stock_id;
```

```
    INSERT INTO trade_transactions
    VALUES (p_trans_id, p_investor_id, p_stock_id,
            p_portfolio_id, 'BUY', p_qty, v_price, SYSDATE);

    DBMS_OUTPUT.PUT_LINE('BUY ORDER SUCCESSFUL');
END;
/
```

Stored Procedure — sell_stock:

Performs the corresponding SELL operation. The trigger trg_check_sell validates the requested quantity against current holdings before this insert is committed.

```
CREATE OR REPLACE PROCEDURE sell_stock(
    p_trans_id     NUMBER,
    p_investor_id  NUMBER,
    p_stock_id     NUMBER,
    p_portfolio_id NUMBER,
    p_qty          NUMBER
)
IS
    v_price NUMBER;
BEGIN
    SELECT current_price INTO v_price
    FROM stock
    WHERE stock_id = p_stock_id;

    INSERT INTO trade_transactions
    VALUES (p_trans_id, p_investor_id, p_stock_id,
```

```
            p_portfolio_id, 'SELL', p_qty, v_price, SYSDATE);


    DBMS_OUTPUT.PUT_LINE('SELL ORDER SUCCESSFUL');
END;
/
```

Cursor-Based Procedure — portfolio_report:

Uses an explicit cursor to iterate over a JOIN of portfolios and investors, generating a formatted listing of all investor portfolios.

```
CREATE OR REPLACE PROCEDURE portfolio_report
IS
    CURSOR c1 IS
        SELECT p.portfolio_name, i.name
        FROM portfolio p
        JOIN investor i ON p.investor_id = i.investor_id;
BEGIN
    FOR rec IN c1 LOOP
        DBMS_OUTPUT.PUT_LINE(rec.name || ' -> ' ||
                             rec.portfolio_name);
    END LOOP;
END;
/
```

Trigger — trg_update_holdings:

Automatically updates the holdings table whenever a new trade is recorded. For BUY transactions, it inserts a new holding row or increments the existing quantity. For SELL transactions, it decrements the quantity.

```
CREATE OR REPLACE TRIGGER trg_update_holdings
AFTER INSERT ON trade_transactions
```

```
FOR EACH ROW

DECLARE

    v_count NUMBER;

BEGIN

    SELECT COUNT(*) INTO v_count

    FROM holdings

    WHERE portfolio_id = :NEW.portfolio_id

      AND stock_id = :NEW.stock_id;



    IF :NEW.trans_type = 'BUY' THEN

        IF v_count = 0 THEN

            INSERT INTO holdings VALUES (

                holdings_seq.NEXTVAL,

                :NEW.portfolio_id, :NEW.stock_id,

                :NEW.quantity, :NEW.price);

        ELSE

            UPDATE holdings

            SET total_quantity = total_quantity + :NEW.quantity

            WHERE portfolio_id = :NEW.portfolio_id

              AND stock_id = :NEW.stock_id;

        END IF;

    ELSIF :NEW.trans_type = 'SELL' THEN

        UPDATE holdings

        SET total_quantity = total_quantity - :NEW.quantity

        WHERE portfolio_id = :NEW.portfolio_id

          AND stock_id = :NEW.stock_id;

    END IF;
```

```
END;

/
```

Trigger — trg_check_sell:

A BEFORE INSERT trigger that prevents overselling — raises an application error if an investor attempts to sell more shares than currently held.

```
CREATE OR REPLACE TRIGGER trg_check_sell

BEFORE INSERT ON trade_transactions

FOR EACH ROW

DECLARE

    v_qty NUMBER;

BEGIN

    IF :NEW.trans_type = 'SELL' THEN

        SELECT total_quantity INTO v_qty

        FROM holdings

        WHERE portfolio_id = :NEW.portfolio_id

          AND stock_id = :NEW.stock_id;



        IF v_qty < :NEW.quantity THEN

            RAISE_APPLICATION_ERROR(-20001,

                'Not enough shares to sell');

        END IF;

    END IF;

END;

/
```

Trigger — trg_log_trade:

An audit trigger that records every transaction into the admin_log table, providing a complete trail of all trading activity.

```
CREATE OR REPLACE TRIGGER trg_log_trade

AFTER INSERT ON trade_transactions

FOR EACH ROW

BEGIN

    INSERT INTO admin_log VALUES (

        admin_seq.NEXTVAL,

        'New ' || :NEW.trans_type ||

        ' Transaction by Investor ' || :NEW.investor_id,

        SYSDATE);

END;

/
```

Sequences:

```
CREATE SEQUENCE holdings_seq START WITH 100 INCREMENT BY 1;

CREATE SEQUENCE admin_seq    START WITH 1;
```

View — vw_portfolio_summary:

Joins investor, portfolio, holdings, and stock tables to compute real-time market value of every holding.

```
CREATE OR REPLACE VIEW vw_portfolio_summary AS

SELECT i.investor_id, i.name, p.portfolio_name,

       s.symbol, h.total_quantity, s.current_price,

       (h.total_quantity * s.current_price) AS market_value

FROM investor i

JOIN portfolio p ON i.investor_id = p.investor_id

JOIN holdings  h ON p.portfolio_id = h.portfolio_id

JOIN stock     s ON h.stock_id = s.stock_id;
```

9. Sample SQL Queries

A representative set of queries was implemented to demonstrate the analytical capabilities of the database:

1. Total market value per investor:

```
SELECT name, SUM(market_value) AS total_value

FROM vw_portfolio_summary

GROUP BY name;
```

2. Highest value portfolio:

```
SELECT * FROM (

    SELECT name, SUM(market_value) total_value

    FROM vw_portfolio_summary

    GROUP BY name

    ORDER BY total_value DESC

)

WHERE ROWNUM = 1;
```

3. Investors holding IT-sector stocks:

```
SELECT DISTINCT i.name

FROM investor i

JOIN portfolio p ON i.investor_id = p.investor_id

JOIN holdings  h ON p.portfolio_id = h.portfolio_id

JOIN stock     s ON h.stock_id = s.stock_id

WHERE s.sector = 'IT';
```

4. Sector-wise average price:

```
SELECT sector, ROUND(AVG(current_price),2) avg_price

FROM stock
```

```
GROUP BY sector;
```

5. Stocks never traded:

```
SELECT stock_name FROM stock
WHERE stock_id NOT IN (
    SELECT DISTINCT stock_id
    FROM trade_transactions
);
```

6. Investors with more than one trade:

```
SELECT investor_id, COUNT(*) total_trades
FROM trade_transactions
GROUP BY investor_id
HAVING COUNT(*) > 1;
```

7. Portfolios with current value greater than 20,000:

```
SELECT name, SUM(market_value) total_value
FROM vw_portfolio_summary
GROUP BY name
HAVING SUM(market_value) > 20000;
```

10. Tools & Technologies Used

DBMS Software: Oracle Database (compatible with Oracle 11g and above)

Programming Languages: SQL, PL/SQL

Tools: Oracle SQL Developer, SQL*Plus

Version Control: Git (GitHub repository for collaborative development)

11. Expected Outcomes

1. Successful creation of a fully functional, normalized database for a stock trading and portfolio management system.
2. Efficient data retrieval and management using SQL joins, views, and aggregate queries.
3. Automation of holdings updates and audit logging through database triggers.
4. Prevention of invalid trades (e.g., overselling) using BEFORE INSERT triggers and check constraints.
5. High data consistency and integrity through enforcement of foreign key constraints, unique constraints, and 3NF normalization.
6. Demonstration of core PL/SQL constructs — procedures, triggers, cursors, sequences, and views — applied to a realistic financial domain.